

Generative Adversarial Networks (GANs)

Conditional GANs (cGANs)

The limitation of a vanilla GAN

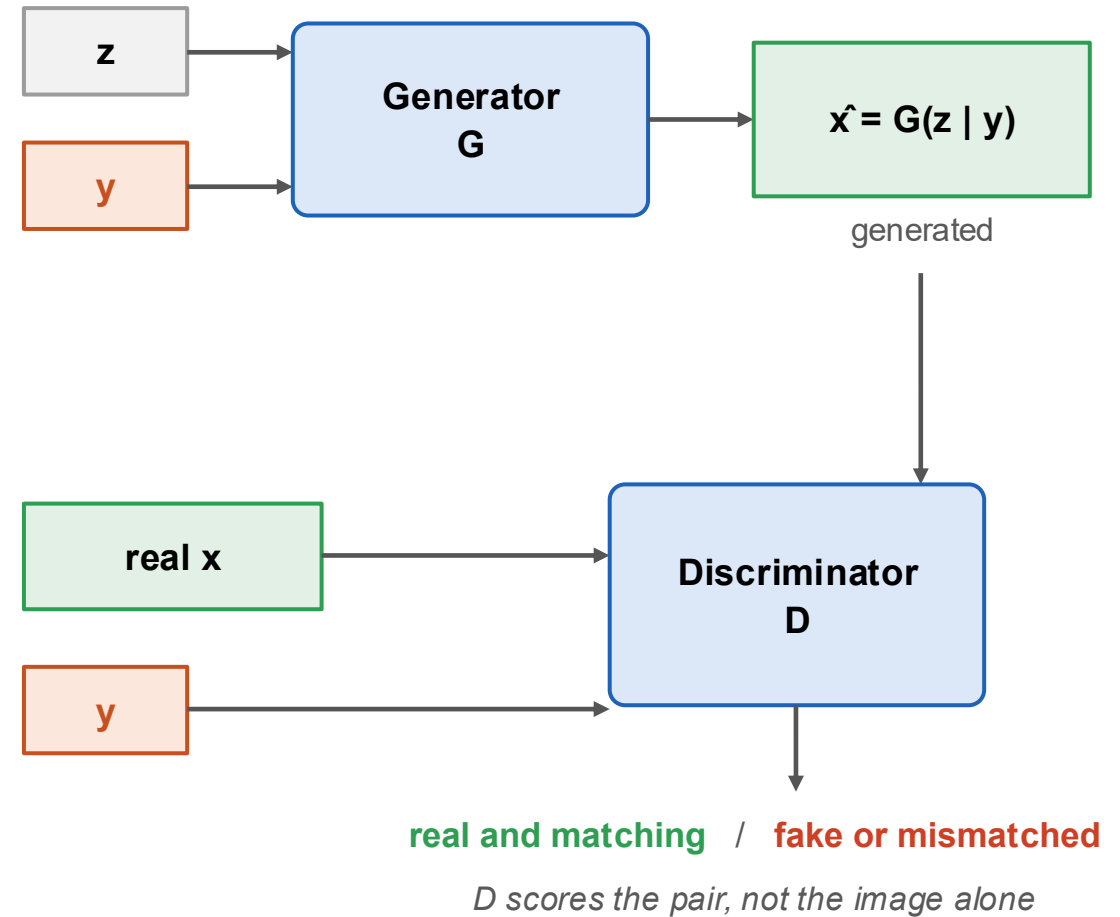
- $z \rightarrow G$ gives you a sample from the data distribution — but you cannot ask for a particular one.
- Train on MNIST and you get digits; you cannot request a 7.

The fix: condition both networks

- Feed a condition y to G and to D — a class label, a sentence, or another image.
- D no longer asks “is x real?” but “is the pair (x, y) real and matching?”
- Mismatched real pairs (real image, wrong label) can be fed to D as negatives.

$$\min_G \max_D E[\log D(x | y)] + E[\log(1 - D(G(z | y)))]$$

the only change from the GAN objective is “ $| y$ ” in both terms



cGANs: how the condition gets into the network

1 Concatenate to the input vector

One-hot the label, append it to z , and let the first dense layer mix them. Simplest option; standard for MLP or DCGAN generators.



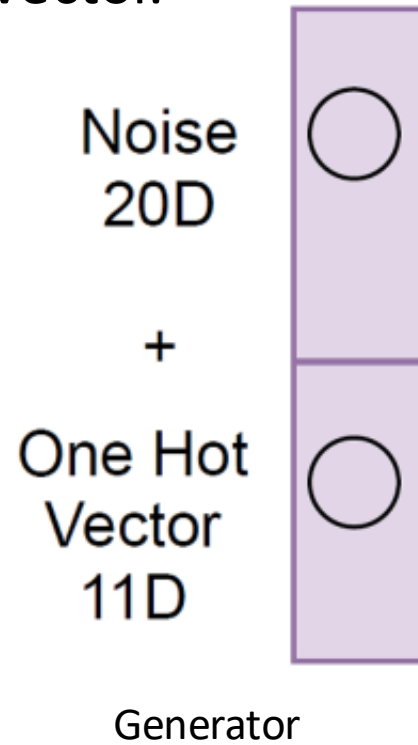
The two failure modes to watch for

- The generator ignores y — samples look fine but do not match the condition. Check by generating a full row per class, not one image.
- The discriminator only checks y and stops looking at x — the generator then only has to get the label right, and image quality collapses.

Diagnostic: fix z and sweep y across all classes. If the row does not change, the generator is ignoring the condition.

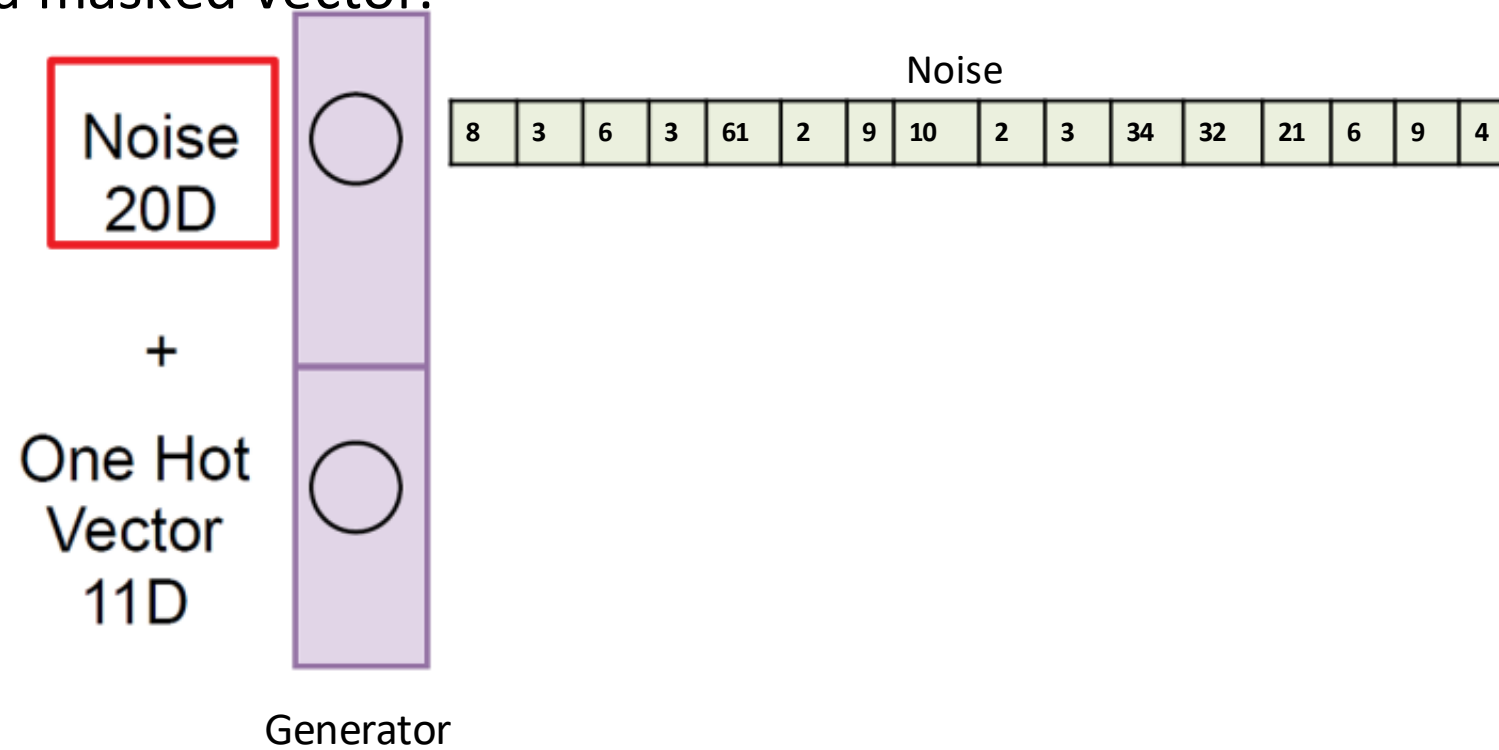
Detailed CGAN Architecture

For each quarter , input of the generator is a noise and a one hot vector.

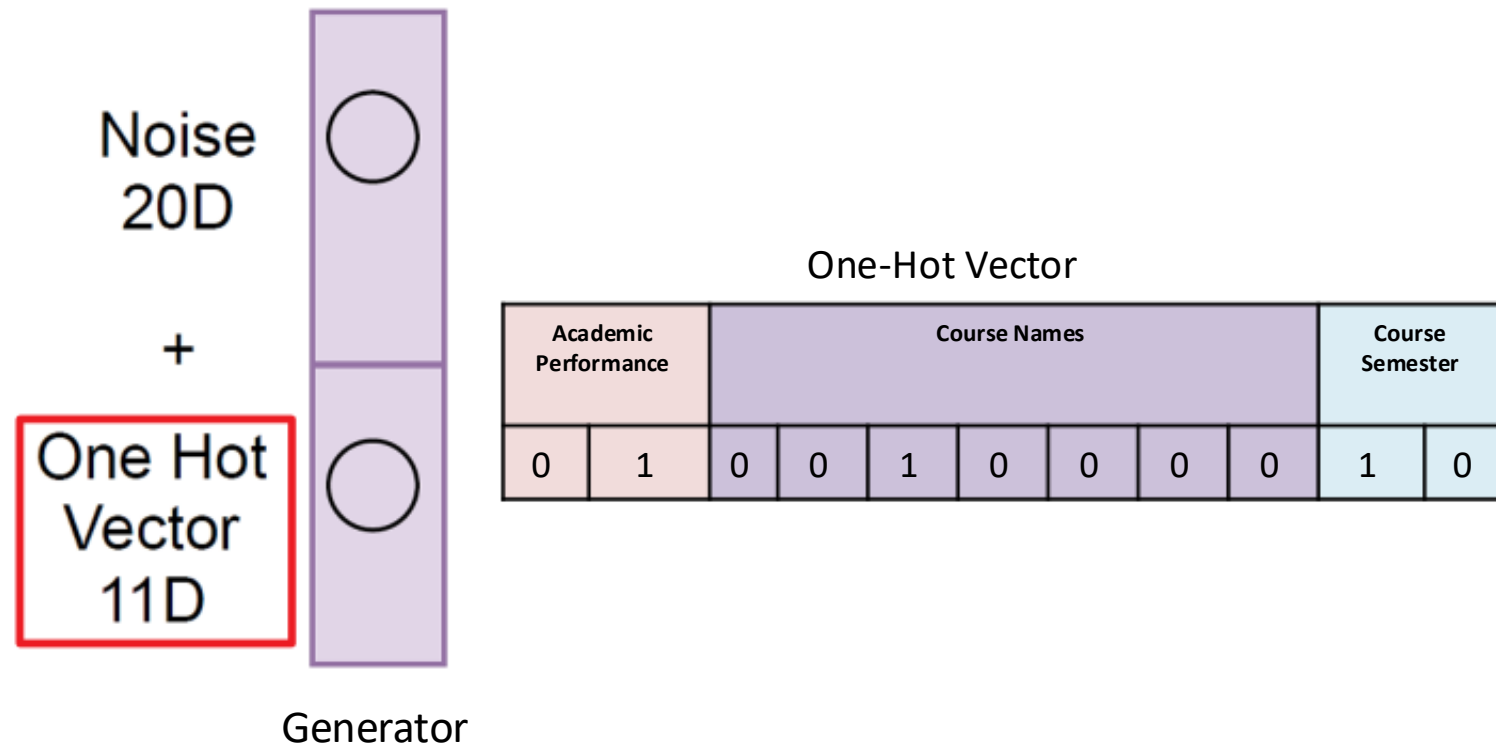


Detailed CGAN Architecture

For each quarter input of the generator is a noise and a masked vector.



Detailed CGAN Architecture



Different combinations of these features, form a one-hot vector, hence various combinations exist in the dataset.

Detailed CGAN Architecture

8	3	6	3	61	2	9	10	2	3	34	32	21	6	9	4
---	---	---	---	----	---	---	----	---	---	----	----	----	---	---	---

0	1	0	0	1	0	0	0	0	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---

Generator

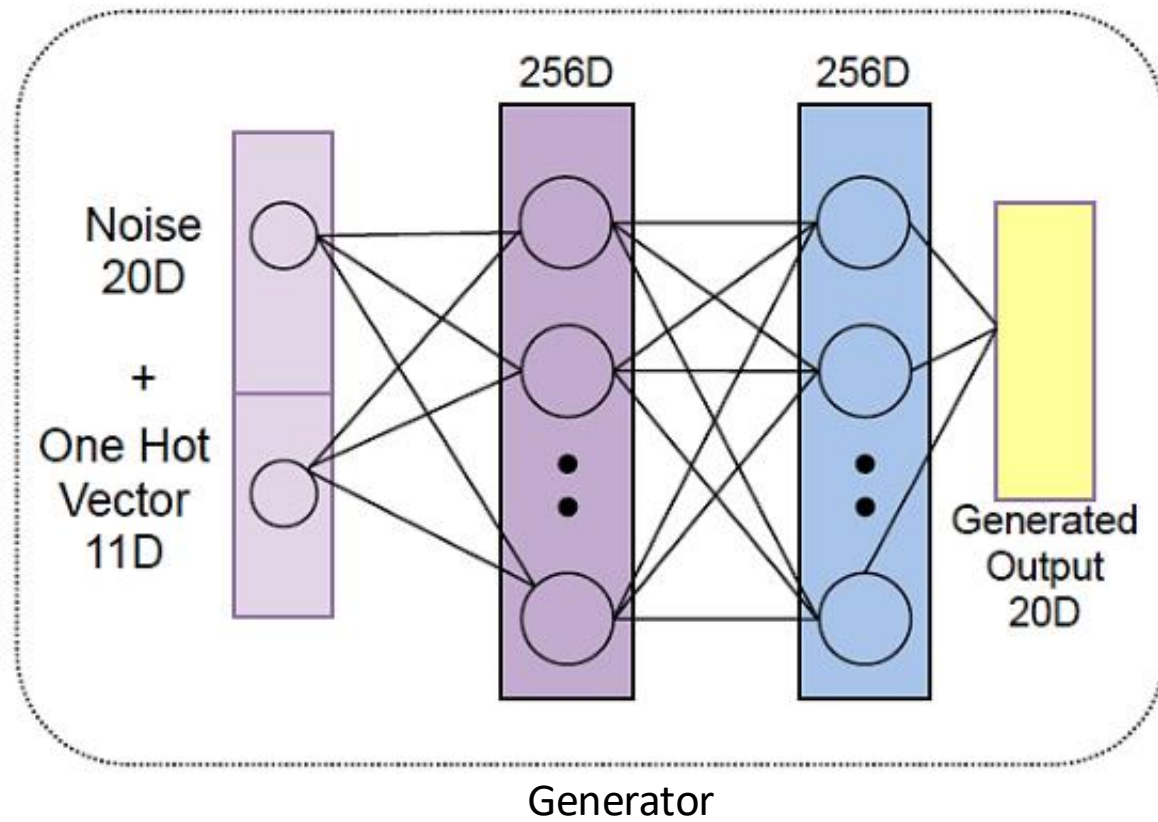
Noise
20D

+

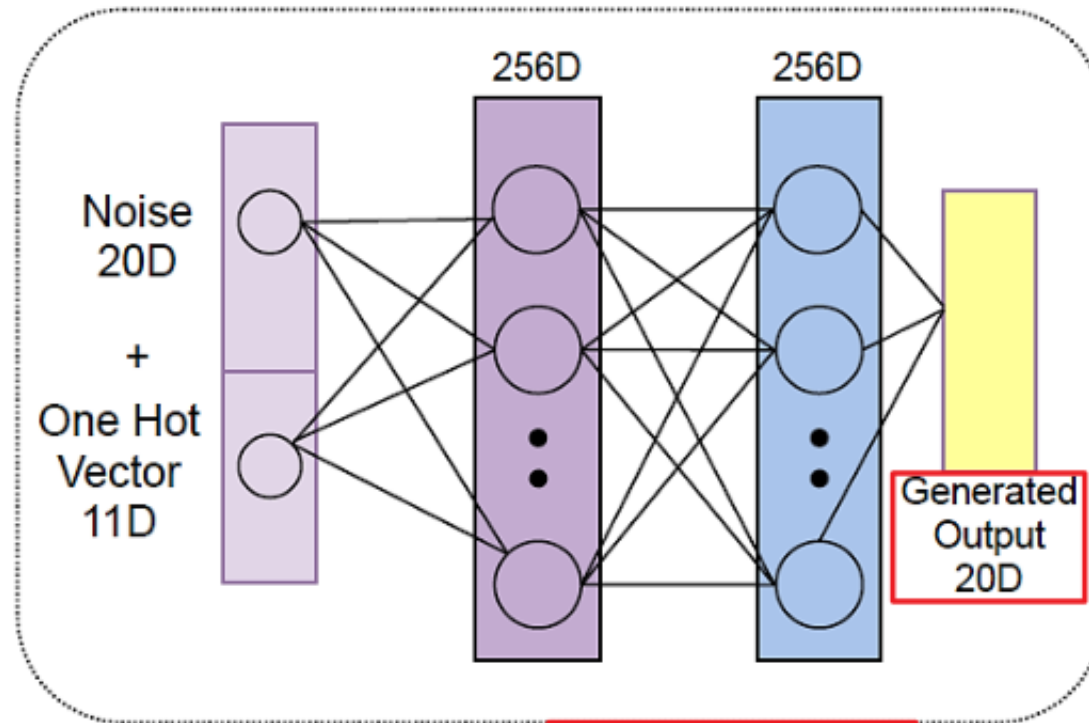
One Hot
Vector
11D



Detailed CGAN Architecture



Detailed CGAN Architecture

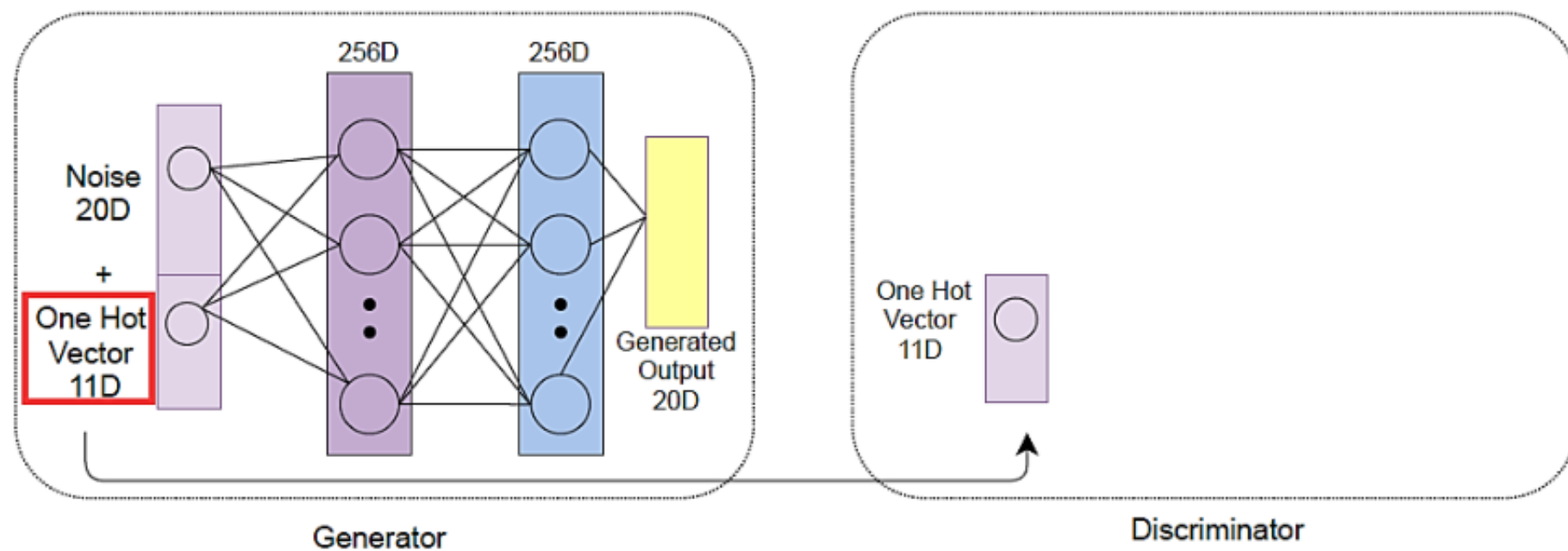


For each quarter output of the generator is a vector of 20 dimension.

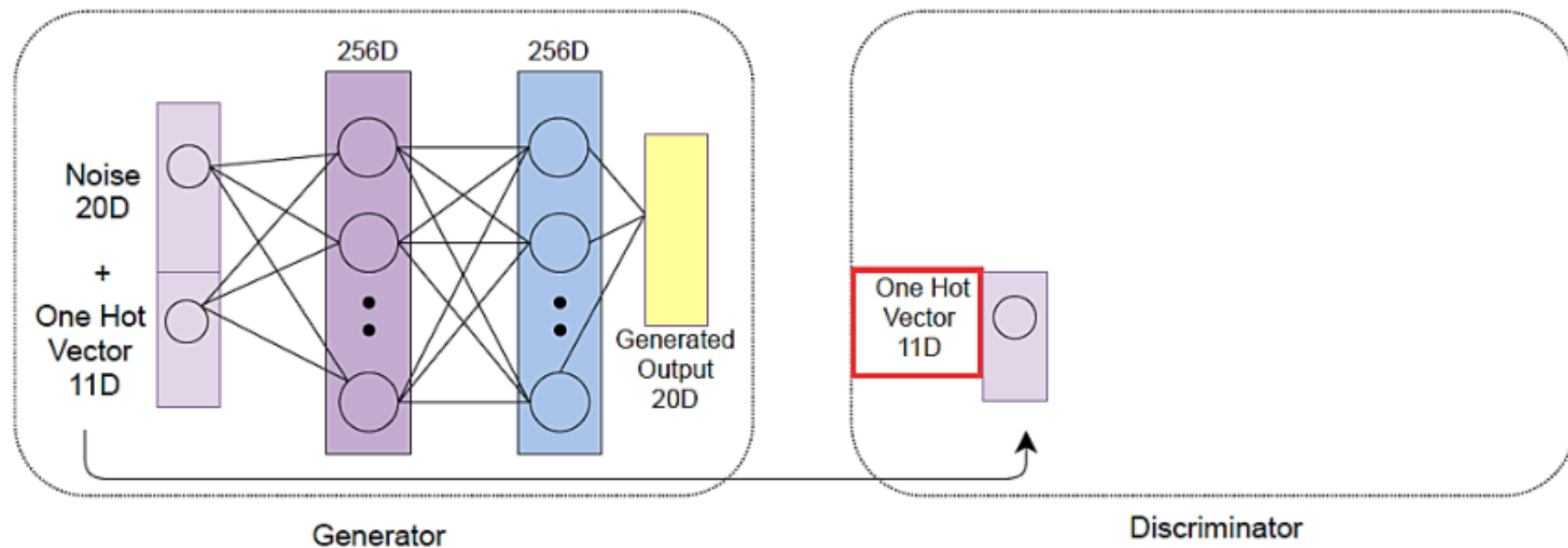
Generator's Output

9	2	41	12	8	9	0	5	43	7	21	8	6	44	9	0	7	5	44	3
---	---	----	----	---	---	---	---	----	---	----	---	---	----	---	---	---	---	----	---

Detailed CGAN Architecture



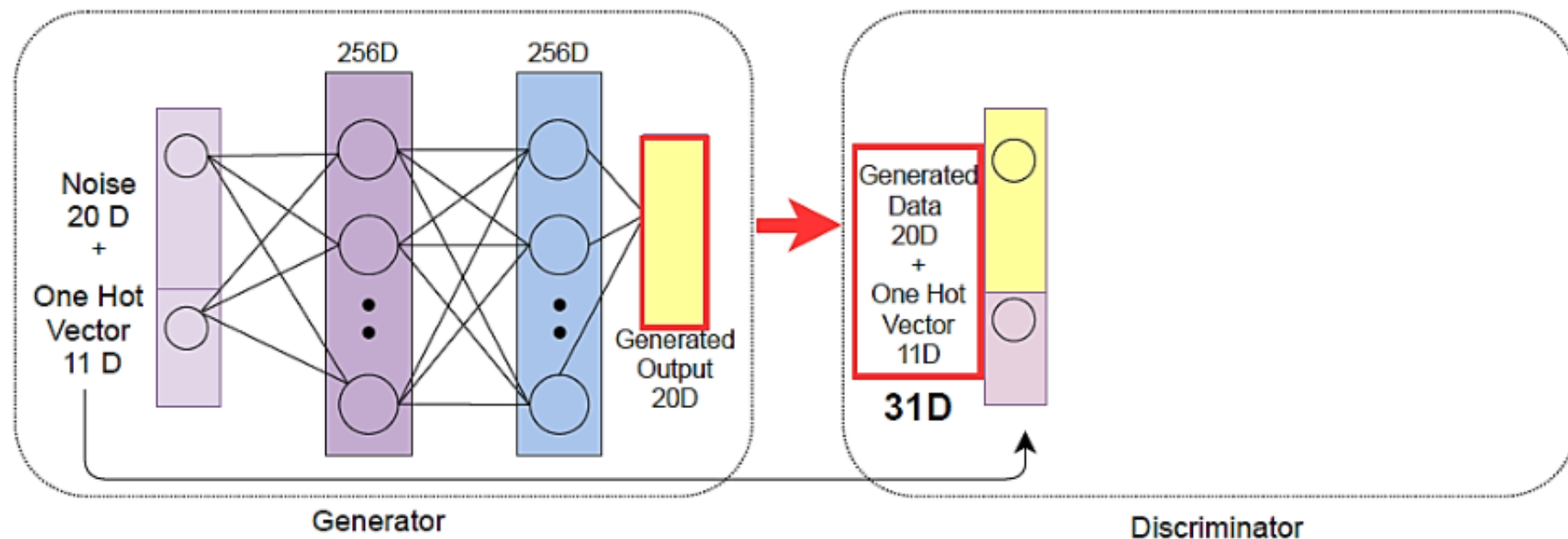
Detailed CGAN Architecture



Detailed CGAN Architecture

The discriminator also takes the condition y as input along with the sample (real or generated).

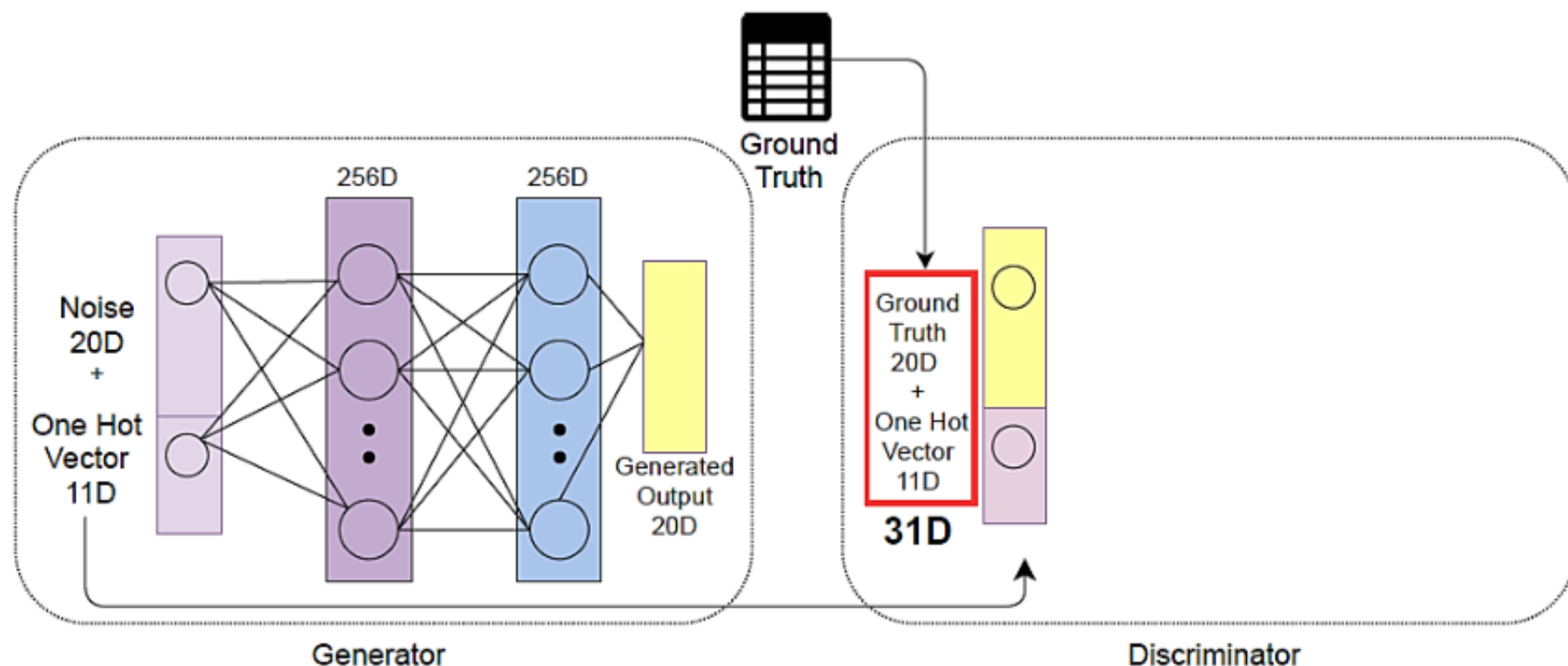
It learns to distinguish real samples x from generated samples $G(z|y)$, given the context y .



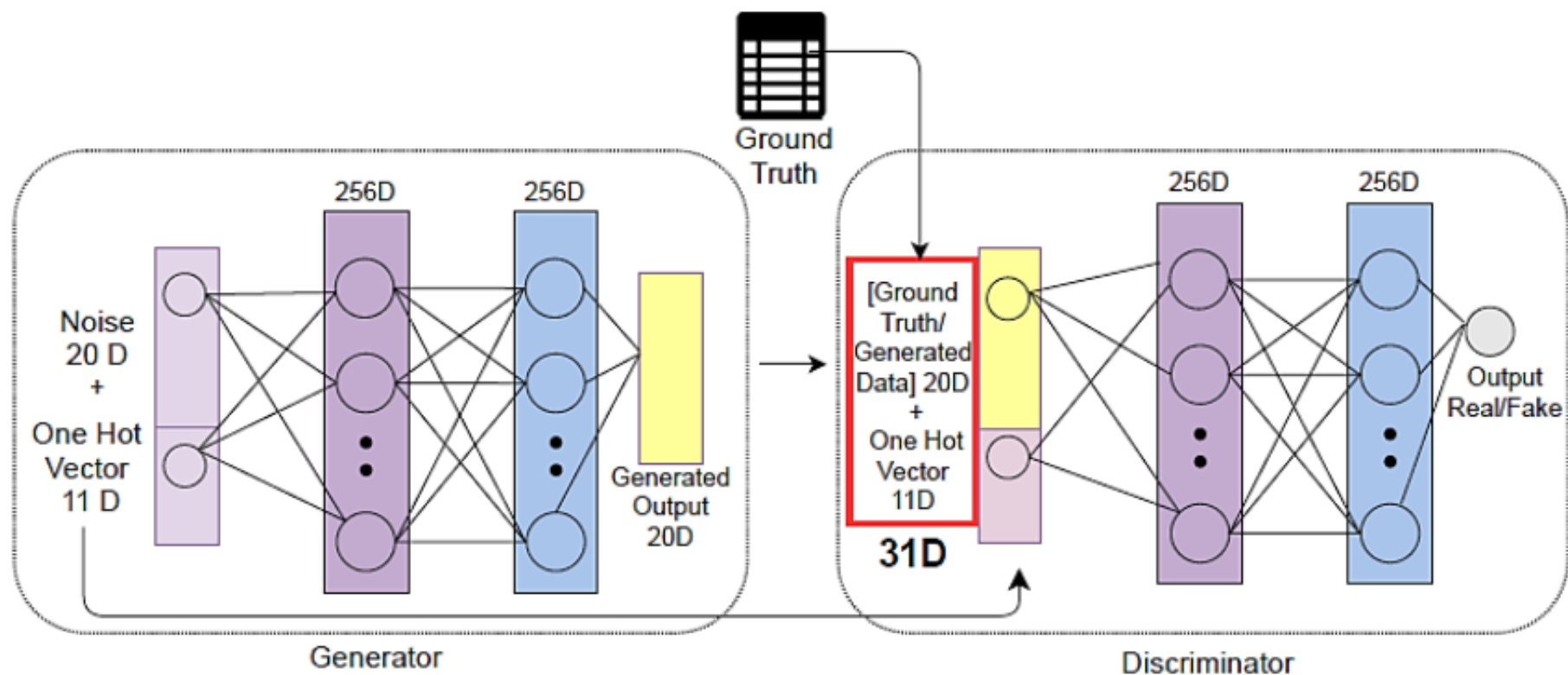
Detailed CGAN Architecture

The discriminator also takes the condition y as input along with the sample (real or generated).

It learns to distinguish real samples x from generated samples $G(z|y)$, given the context y .



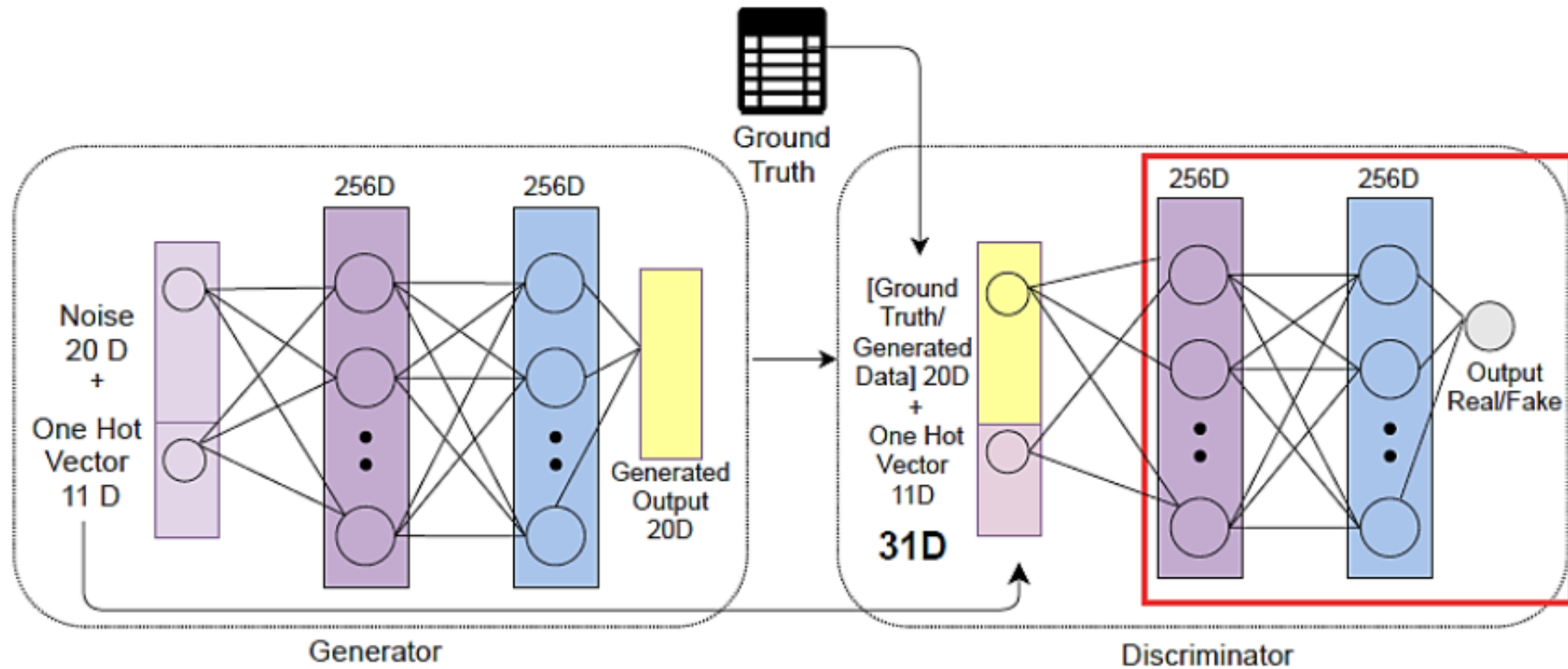
Detailed CGAN Architecture



Detailed CGAN Architecture

The discriminator also takes the condition y as input along with the sample (real or generated).

It learns to distinguish real samples x from generated samples $G(z|y)$, given the context y .



In case you want to know more about Conditional GAN, this is great short [article](#) from [Connor Shorten](#)

cGANs: what the condition can be

Class label

one-hot vector

→ Class-conditional generation — ask for a 7, or for “tabby cat”

An image

the input image itself

→ Image-to-image translation: pix2pix (edges → photo, map → aerial, day → night)

A sentence

text embedding $\phi(t)$

→ Text-to-image synthesis — and the reason we need StackGAN next

Where this runs out

- Conditioning tells the generator what to draw. It does not make drawing it any easier.
- A single generator asked for 256×256 photo-realistic images from text trains unstably and produces blurry, low-diversity samples.

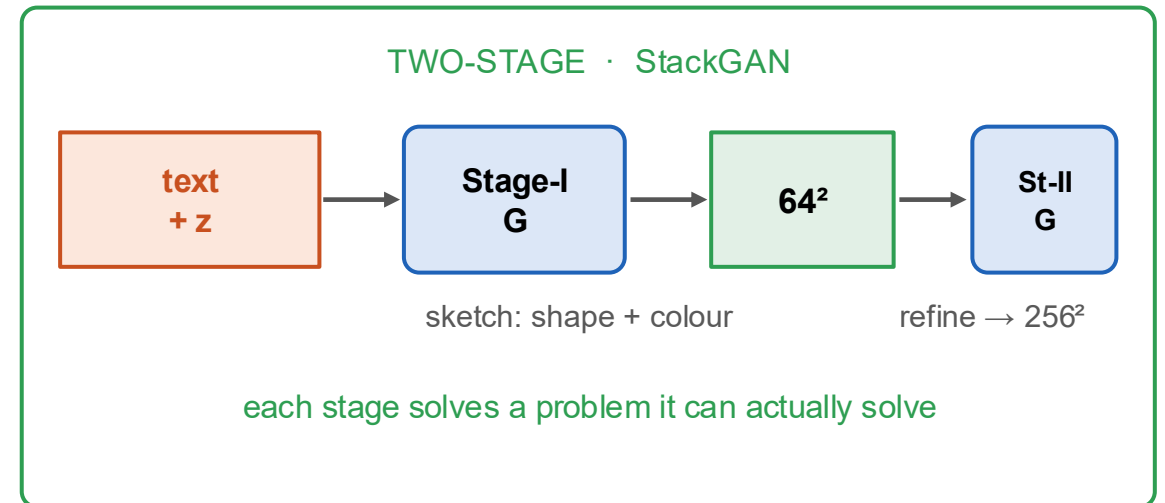
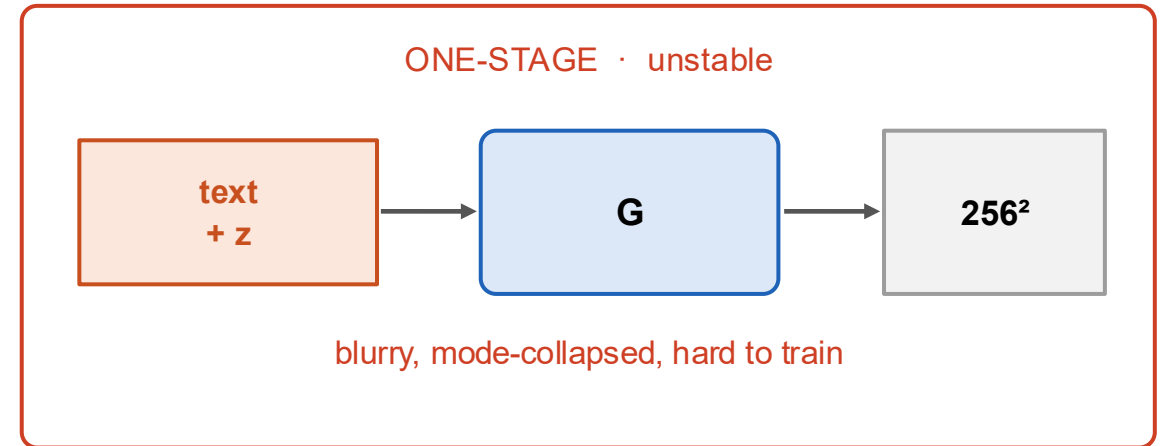
StackGAN: why stack two GANs?

The problem

- Text $\rightarrow 256 \times 256$ in one step asks the generator to invent shape, colour, pose and fine texture simultaneously.
- The natural image manifold and the model distribution rarely overlap at high resolution, so the discriminator wins easily and training destabilises.

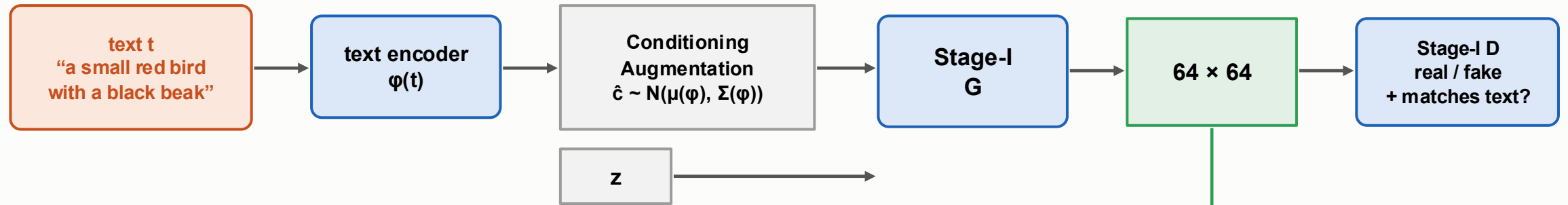
The idea

- Split one hard problem into two easy ones — sketch, then refine.
- Stage-I only has to get rough shape and colour right, at 64×64 .
- Stage-II starts from that sketch, so it never faces a blank canvas — it corrects defects and adds detail.



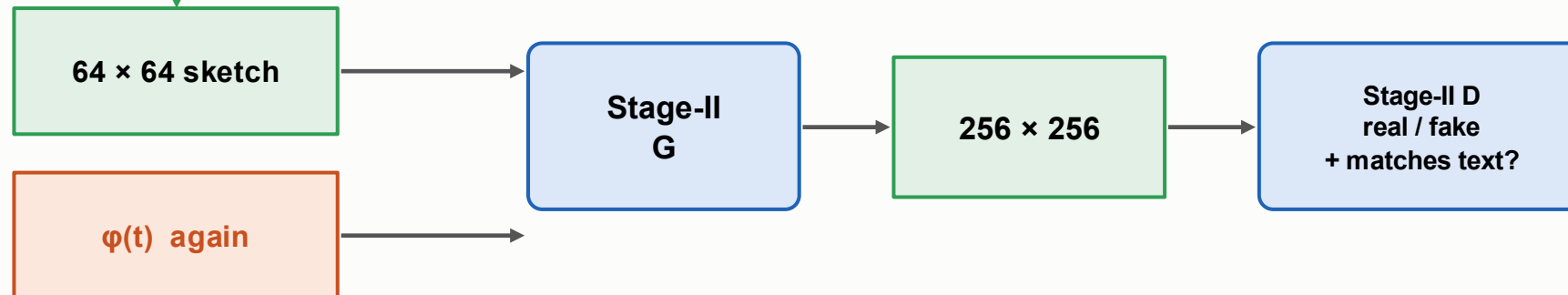
StackGAN: the two-stage architecture

STAGE-I · sketch the primitive shape and colours



the 64 × 64 sketch is the input to Stage-II

STAGE-II · correct defects and add detail

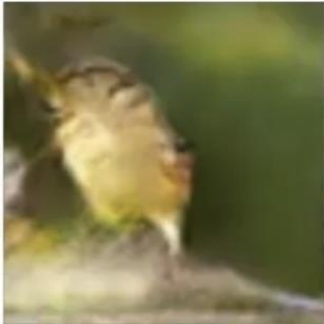


No fresh z in Stage-II. The randomness entered at Stage-I; Stage-II is a refinement network. It re-reads $\varphi(t)$ so details the sketch missed can still be added.

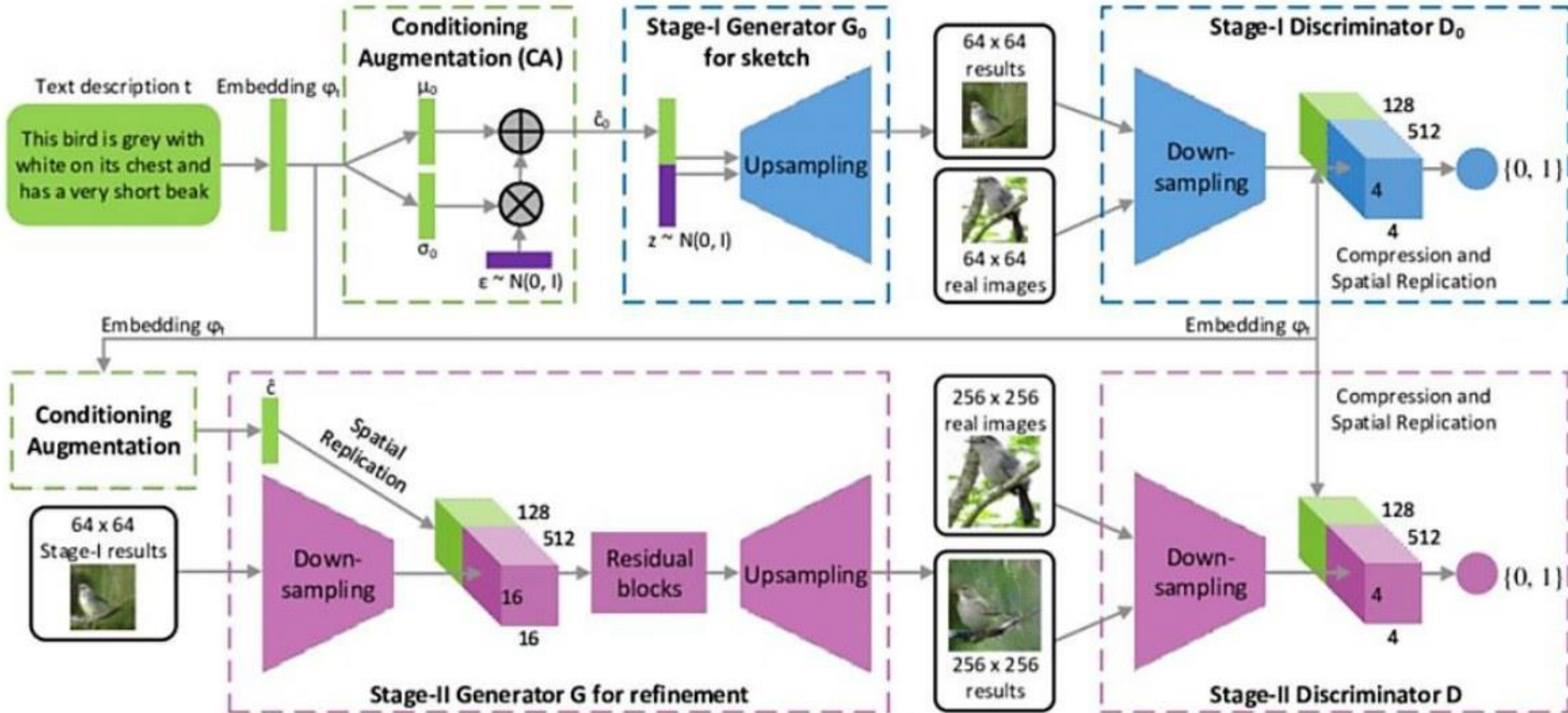
Stacked Generative Adversarial Networks (StackGAN) is able to generate 256×256 photo-realistic images conditioned on text descriptions.

- The Stage-I GAN sketches the primitive shape and basic colors of the object based on the given text description, yielding Stage-I low resolution images.
- The Stage-II GAN takes Stage-I results and text descriptions as inputs, and generates high resolution images with photo-realistic details.”

Stacked Generative Adversarial Networks (StackGAN) is able to generate 256×256 photo-realistic images conditioned on text descriptions.

Text description	This bird is blue with white and has a very short beak	This bird has wings that are brown and has a yellow belly	A white bird with a black crown and yellow beak	This bird is white, black, and brown in color, with a brown beak	The bird has small beak, with reddish brown crown and gray belly	This is a small, black bird with a white breast and white on the wingbars.	This bird is white black and yellow in color, with a short black beak
Stage-I images							
Stage-II images							

Stack GAN



GANs vs VAEs

- Pros and Cons of GANs:**

- Pros:** High-quality sample generation, especially for images, and ability to model complex data distributions.

- Cons:** Training instability, mode collapse, and difficulty in generating structured data (like text).

- VAEs:**

- Pros:** Easier to train, provide a continuous latent space (which is useful for interpolation)

- Cons:** Generated samples often lack sharp details and can be blurry compared to GANs.

Evaluating GANs: why it is hard

There is no likelihood to report

- A GAN gives you a sampler, not a density — you cannot quote test log-likelihood the way you can for a VAE or PixelCNN.
- Human judgement works but is slow, expensive and drifts between annotators and sessions.
- So we need an automatic number. It has to capture two things at once:

FIDELITY
is each sample realistic?

DIVERSITY
does it cover the data?

Either one alone is trivial to cheat. A generator that memorises one perfect training image has perfect fidelity and no diversity; one that outputs noise has diversity and no fidelity.

Sharp but narrow

every sample is a crisp, well-formed image — but they are all nearly the same

Diverse but blurry

the whole data range is covered, but nothing looks real

Memorised the training set

flawless and varied samples — copied straight from the data

A single number has to punish all three. IS punishes one of them; FID punishes all three.

INTUITION

Inception Score: The central idea

A good generator should produce images that a classifier is confident about, while the collection as a whole should contain many different classes.

PER IMAGE

Confident, recognisable predictions

+

ACROSS THE SET

Predictions spread over many classes

= one score

METHOD

The setup: borrow a pretrained classifier

1

Generate an image

$x = G(z)$ from your trained generator

2

Feed it to Inception

a classifier pretrained on ImageNet — never trained on your images

3

Read off the distribution

a probability for every class the network knows

$$p(y|x)$$

probability that image x
belongs to class y

The classifier is fixed. We are borrowing its opinion, not adapting it to our data.

EXAMPLE

What the classifier gives back

For one generated image, Inception returns a probability for each class:

Class

dog

cat

car

bird

horse

Probability

0.01

0.02

0.01

0.94

0.02

“I am very
confident this is
a bird.”

$p(y|x)$

written as a conditional distribution: the class distribution given this particular image

QUALITY

Property 1 — each image should be recognisable

Compare two generated images and what the classifier says about each.

IMAGE A — CONFIDENT

$$p(y|x) = [0.99, 0.003, 0.002, 0.005]$$

The distribution is sharply peaked. Low entropy.
The image clearly contains one recognisable object.

IMAGE B — UNCERTAIN

$$p(y|x) = [0.25, 0.25, 0.25, 0.25]$$

The distribution is flat. High entropy. The classifier cannot tell what it is looking at.

So the first half of the metric rewards low entropy of $p(y|x)$ — classifier confidence on individual images.

REFRESHER

Entropy, briefly

Entropy measures uncertainty in a distribution.

$$H(Y) = -\sum_y p(y) \log p(y)$$

LOW ENTROPY

[0.99, 0.003, 0.002, 0.005] — sharply concentrated, almost no uncertainty

HIGH ENTROPY

[0.25, 0.25, 0.25, 0.25] — spread evenly, maximum uncertainty

THE CATCH

But confidence alone is not enough

Imagine a deliberately terrible generator that produces only dogs — but very clear, very convincing dogs.

dog

dog

dog

dog

dog

dog

dog

Every image scores perfectly on quality: $p(y|x) \approx [1, 0, 0, \dots]$ every single time.

MODE COLLAPSE

A metric built only on confidence would give this generator top marks. We need a second term that penalises repetition.

DIVERSITY

Property 2 — the set should cover many classes

Generate N images, then average the classifier predictions across all of them:

$$p(y) \approx \frac{1}{N} \sum_{i=1}^N p(y|x_i)$$

This is the marginal class distribution — what the generator produces on average, ignoring any individual image.

READING IT

If $p(y)$ is concentrated on one class, the generator is producing one thing. If $p(y)$ is spread out, the generator covers the space.

COMPARISON

Two generators, identical confidence

COLLAPSED GENERATOR

every image $\rightarrow$ dog

$$p(y) = [1, 0, 0, 0]$$

$$H(Y) = 0$$

no diversity at all

DIVERSE GENERATOR

dog · cat · car · aeroplane · horse ...

$$p(y) \approx [0.2, 0.2, 0.2, 0.2, 0.2]$$

$$H(Y) = \log K$$

spread across the class space

So the second half of the metric rewards high entropy of the marginal $p(y)$.

Inception Score: the formula

$$\text{IS}(G) = \exp(E_{x \sim p_g} [D_{\text{KL}}(p(y | x) \parallel p(y))])$$

Where each piece comes from

- $p(y | x)$ — the classifier's label distribution for one generated image.
- $p(y) = E[p(y | x)]$ — the same distribution averaged over all the samples.
- Higher is better. The exp only makes the numbers easier to read.

BUILDING BLOCK

KL divergence

KL divergence measures how different two probability distributions are.

$$D_{KL}(P \parallel Q) = \sum_y P(y) \log \frac{P(y)}{Q(y)}$$

Applied to our two distributions:

$$D_{KL}(p(y|x) \parallel p(y)) = \sum_y p(y|x) \log \frac{p(y|x)}{p(y)}$$

In words: how different is this one image's class prediction from the class distribution of the whole generated set?

DERIVATION

Why one quantity captures both properties

Expand the definition and take the expectation over generated images:

$$\begin{aligned} D_{KL}(p(y|x) \parallel p(y)) &= \sum_y p(y|x) \log \frac{p(y|x)}{p(y)} \\ &= \sum_y p(y|x) \log p(y|x) - \sum_y p(y|x) \log p(y) \end{aligned}$$

$$E_x[D_{KL}(p(y|x) \parallel p(y))] = H(Y) - H(Y|X)$$

The two terms are exactly the two properties we wanted.

INTERPRETATION

It is mutual information

$$I(X; Y) = H(Y) - H(Y|X)$$

$$IS = e^{I(X; Y)}$$

$H(Y)$ — DIVERSITY

How much variety is there in the predicted classes across the whole generated set? We want this HIGH.

$H(Y|X)$ — RECOGNISABILITY

How uncertain is the classifier once it has seen an individual image? We want this LOW.

X = generated images, Y = their predicted classes

RECOGNISABILITY

Reading the first term: $H(Y|X)$

Conditional entropy: how uncertain the classifier remains after seeing one generated image.

A GOOD IMAGE

image → clearly a dog

The classifier commits. Conditional entropy is near zero.

A CONFUSING IMAGE

image → 30% dog, 25% cat, 20% horse, 25% other

The classifier hedges. Conditional entropy is high.

$$H(Y|X) \approx 0$$

Maximising IS therefore pushes $H(Y|X)$ down — every image should be recognisable.

DIVERSITY

Reading the second term: $H(Y)$

The entropy of the class distribution across all generated images.

ONLY DOGS

$$p(y) = [1, 0, 0, \dots]$$

Entropy is zero. The generator has collapsed onto a single class.

MANY CLASSES, EVENLY

$$p(y) = [1/K, 1/K, \dots, 1/K]$$

Entropy is $\log K$ — the maximum possible for K classes.

$$H(Y) = \log K$$

Maximising IS therefore pushes $H(Y)$ up — the set should cover many different classes.

NUMBERS

Worked example — a good generator

A toy classifier with three classes: {cat, dog, car}. The generator produces three images.

Image 1 [1, 0, 0] *clearly a cat*

Image 2 [0, 1, 0] *clearly a dog*

Image 3 [0, 0, 1] *clearly a car*

$$H(Y|X) = 0 \quad H(Y) = \log 3$$

The maximum possible score for a three-class classifier.

MARGINAL

$$p(y) = \left[\frac{1}{3}, \frac{1}{3}, \frac{1}{3} \right]$$

every class equally represented

$$IS = e^{\log 3} = 3$$

NUMBERS

Worked example — a collapsed generator

Same classifier, but now all three images are cats.

Image 1 [1, 0, 0] *clearly a cat*

Image 2 [1, 0, 0] *clearly a cat*

Image 3 [1, 0, 0] *clearly a cat*

$$H(Y|X) = 0 \quad H(Y) = 0$$

MARGINAL

$$p(y) = [1, 0, 0]$$

one class carries everything

$$IS = e^{0-0} = 1$$

Perfect classifier confidence does NOT guarantee a high Inception Score. The generator also has to be diverse.

INTERPRETATION

What the numbers mean

$$1 \leq IS \leq K$$

- The minimum is 1, because KL divergence is never negative — so the exponent is never below zero.
- The maximum is K, the number of classes the classifier knows. For ImageNet, K = 1000.
- IS close to 1 means the metric sees little class-based information — it is not proof that the images are terrible.
- Values depend on the exact Inception weights, preprocessing and implementation.

CAUTION

Do not compare IS values across papers unless the evaluation protocol is identical.

DETAIL

Why exponentiate at all?

We could simply report the mutual information. The exponential converts a quantity measured in log units into something easier to read as an effective number of classes.

$$I(X; Y) = \log 10 \Rightarrow IS = 10$$

A score of 10 can be read as: the generated set carries about as much class information as a perfectly balanced set of 10 distinct classes.

ALGORITHM

Computing it in practice

- 1 Generate N images from your model — typically N = 50,000.
- 2 Run every image through the pretrained Inception network to get its class distribution.
- 3 Average those vectors to obtain the marginal $p(y)$.
- 4 Compute the KL divergence between $p(y | x_i)$ and $p(y)$ for every image.
- 5 Take the mean of those N divergences.
- 6 Exponentiate.

$$IS = \exp \left(\frac{1}{N} \sum_{i=1}^N D_{KL}^{(i)} \right)$$

the whole algorithm

PRACTICE

Why implementations use splits

Most implementations do not compute a single score. They divide the generated images into groups and score each group separately.

50,000 generated images → 5 splits → five scores

10,000

IS1

10,000

IS2

10,000

IS3

10,000

IS4

10,000

IS5

REPORTED AS

mean $\pm$ standard deviation — for example,
 7.42 ± 0.12

WHY

The spread across splits gives some indication of statistical variability in the estimate.

Calculating the Inception Score

Four generated images, three classes, worked through by hand

The question

A generative model produces four images. An Inception classifier returns these class probabilities. Use natural logarithms throughout.

	CLASS 1	CLASS 2	CLASS 3
Image 1	0.90	0.05	0.05
Image 2	0.80	0.10	0.10
Image 3	0.10	0.80	0.10
Image 4	0.05	0.10	0.85

MARGINAL DISTRIBUTION

$$p(y) = \frac{1}{N} \sum_{i=1}^N p(y|x_i)$$

KL DIVERGENCE

$$\text{KL}(p(y|x) \parallel p(y)) = \sum_y p(y|x) \ln \frac{p(y|x)}{p(y)}$$

$$\text{IS} = \exp\left(\frac{1}{N} \sum_{i=1}^N \text{KL}(p(y|x_i) \parallel p(y))\right)$$

STEP 1

Marginal class distribution

Average each column of the table across the four images.

1

$$\begin{aligned} p(y_1) &= \frac{0.90 + 0.80 + 0.10 + 0.05}{4} = \frac{1.85}{4} = 0.4625 \\ p(y_2) &= \frac{0.05 + 0.10 + 0.80 + 0.10}{4} = \frac{1.05}{4} = 0.2625 \\ p(y_3) &= \frac{0.05 + 0.10 + 0.10 + 0.85}{4} = \frac{1.10}{4} = 0.2750 \end{aligned}$$

THE MARGINAL

$$p(y) = [0.4625 \quad 0.2625 \quad 0.2750]$$

SANITY CHECK

$$0.4625 + 0.2625 + 0.2750 = 1$$

STEP 2

KL divergence: image 1 in full

Compare each image's prediction with the marginal, term by term.

$$KL_1 = 0.90 \ln \frac{0.90}{0.4625} + 0.05 \ln \frac{0.05}{0.2625} + 0.05 \ln \frac{0.05}{0.2750}$$

$$\textcircled{2} \approx 0.90 (0.6657) + 0.05 (-1.6582) + 0.05 (-1.7047)$$

$$\approx 0.5992 - 0.0829 - 0.0852 = 0.4310$$

The first term is positive — class 1 is far more likely for this image than on average. The other two are negative, because this image is less likely than average to be class 2 or class 3.

STEP 2

KL divergence: the other three images

The same calculation for images 2, 3 and 4.

$$KL_2 = 0.80 \ln \frac{0.80}{0.4625} + 0.10 \ln \frac{0.10}{0.2625} + 0.10 \ln \frac{0.10}{0.2750} \approx 0.2407$$

$$KL_3 = 0.10 \ln \frac{0.10}{0.4625} + 0.80 \ln \frac{0.80}{0.2625} + 0.10 \ln \frac{0.10}{0.2750} \approx 0.6372$$

$$KL_4 = 0.05 \ln \frac{0.05}{0.4625} + 0.10 \ln \frac{0.10}{0.2625} + 0.85 \ln \frac{0.85}{0.2750} \approx 0.7515$$

Image 4 scores highest: it is the most confident image, and it sits on the class the marginal favours least.

STEP 2 — SUMMARY

The four divergences

KL_1

0.4310

KL_2

0.2407

KL_3

0.6372

KL_4

0.7515

WHAT THE SPREAD TELLS YOU

Every value is positive, as KL divergence must be. Larger values mean the image's prediction differs more from the average prediction — that is, the image carries more class information.

STEP 3

Average the divergences

Take the mean of the four values.

$$3 \quad \overline{\text{KL}} = \frac{0.4310 + 0.2407 + 0.6372 + 0.7515}{4} = \frac{2.0604}{4}$$

AVERAGE KL

$$\overline{\text{KL}} \approx 0.5151$$

This is the exponent in the Inception Score — the expected KL divergence.

STEP 4

Exponentiate

The Inception Score is the exponential of the average KL divergence.

COMPUTE

4

$$IS = e^{\overline{KL}} = e^{0.5151}$$

RESULT

$$IS \approx 1.674$$

WHERE THIS SITS

With three classes the score is bounded by $1 \leq IS \leq 3$. A value of 1.674 says the generator carries real class information, but well short of a perfectly confident, perfectly balanced set.

FINAL ANSWER

$$IS = 1.674$$

AVERAGE KL

0.5151 nats

POSSIBLE RANGE

$$1 \leq IS \leq 3$$

Reported to three decimal places, as the question asks.

INTERPRETATION

What the number means

The Inception Score rewards two things at once, and this example shows both.

CONFIDENCE

Each image has a fairly sharp $p(y|x)$ — one class dominates in every row of the table.

DIVERSITY

Between them the four images cover all three classes, so the marginal $p(y)$ is spread rather than concentrated.

PUTTING IT TOGETHER

Reasonably confident images covering all three classes give an Inception Score of about 1.674 — comfortably above the floor of 1, but short of the ceiling of 3 because the images are not perfectly confident and the classes are not perfectly balanced.

- **IS can be fooled by mode collapse? True?**

CRITICISM

The classifier's label space may not fit

Inception was trained on ImageNet. Its thousand classes are everyday objects. Now consider what you might actually want to generate:

Medical images

Satellite imagery

Human faces

Molecular structures

WHAT GOES WRONG

ImageNet has no meaningful class for a chest X-ray or a benzene ring. $p(y|x)$ becomes poorly calibrated and semantically irrelevant.

WHAT FOLLOWS

The score still computes — it just stops meaning anything about image quality. IS suits domains where the label space is relevant.

CRITICISM

The generator can exploit the classifier

A generator is not obliged to produce images humans find convincing. It only has to produce images the classifier finds recognisable.

HUMAN LOOKING AT THE IMAGE

“This looks weird.”

INCEPTION LOOKING AT THE IMAGE

“99.9% dog!”

IS would happily reward that image.

GENERAL PRINCIPLE

A metric measures the behaviour of its evaluator — not human perception.

CRITICISM

IS says nothing about image-to-image similarity

Suppose your generator produces four images: cat A, cat B, cat C, cat D.

cat A

cat B

cat C

cat D

IS never compares these images with one another. It only looks at their predicted class distributions.

THE CONSEQUENCE

Two visually very different dogs can produce almost identical $p(y|x)$ — so to IS they are interchangeable. Conversely, near-duplicate images are not penalised as long as the classifier keeps assigning them to different classes.

Fréchet Inception Distance

“Do my generated images look like they came from the same distribution as the real images?”

MOTIVATION

FID in one example

You have 10,000 real cat images. Your model generates 10,000 new cat images. Consider two generators.

GENERATOR A

Realistic cats with different poses, colours, backgrounds and sizes.

GENERATOR B

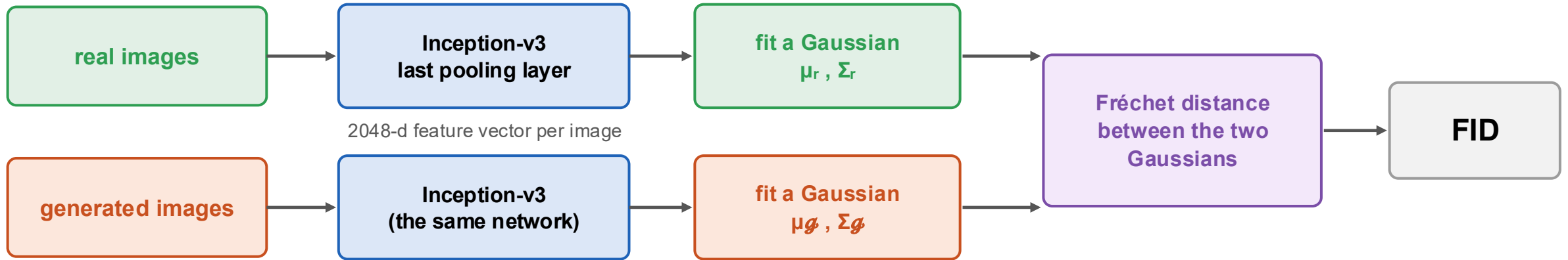
Realistic cats — but almost every cat looks the same.

Both score well on Inception Score: a classifier confidently says “cat” either way.

FID can tell them apart, because it compares the generated collection against the real one.

Fréchet Inception Distance (FID)

Idea: stop reading the classifier's labels. Take its internal features for real and for generated images, and measure how far the two feature distributions are apart.



$$\text{FID} = \| \mu_r - \mu_g \|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

First term: are the features centred in the same place?

A shift in the mean feature vector means the generated images have systematically different content.

Second term: do the features vary in the same way?

This is the term that catches mode collapse — a collapsed generator has far too little variance.

CONCEPT

FID compares distributions, not images

FID never looks at one image on its own. It takes a whole collection of real images and a whole collection of generated images.

FROM THE REAL IMAGES

a feature distribution P_{real}

FROM THE GENERATED IMAGES

a feature distribution $P_{\text{generated}}$

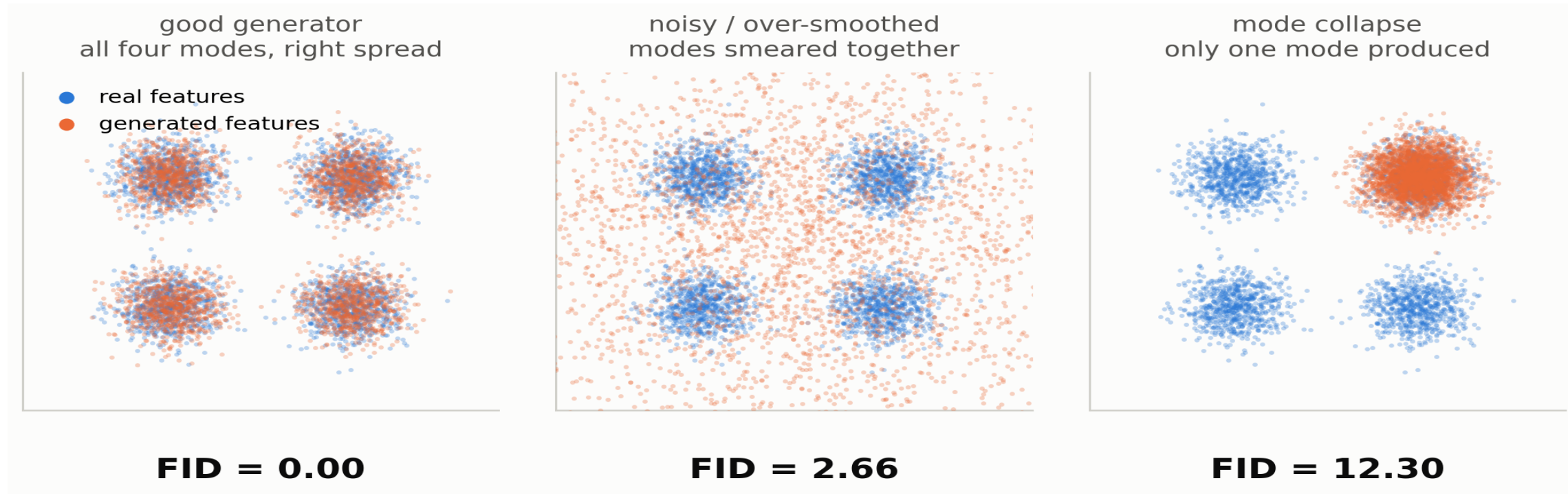
FID asks: how far apart are these two distributions?

Distributions overlap strongly → FID is small

Distributions are far apart → FID is large

What FID rewards and punishes

The same formula on toy 2-D “features”, so you can see the two terms at work:



Mode collapse is punished hardest

Dropping three of four modes moves both the mean and the covariance, so both terms fire at once. This is precisely the failure the Inception Score misses.

Two caveats to state in the same breath

- The Gaussian assumption is a simplification — real feature clouds are not Gaussian.
- FID is biased at small sample sizes: fewer samples inflates it. Use KID when n is small.

METHOD

How FID represents a distribution

FID approximates each cloud with a Gaussian, which is summarised by just two quantities.

MEAN μ

Where is the centre of the distribution?

COVARIANCE Σ

How spread out is it, and how do the feature dimensions vary together?

FOR THE REAL IMAGES

$$\mu_r, \Sigma_r$$

FOR THE GENERATED IMAGES

$$\mu_g, \Sigma_g$$

EXAMPLE

A one-dimensional example

Take a single feature dimension and look at where each distribution is centred.

CENTRES MATCH

$$\mu_r = 10 \quad \mu_g = 10$$

the two distributions sit in the same place — good

CENTRES DIFFER

$$\mu_r = 10 \quad \mu_g = 20$$

the generated distribution is shifted away — FID grows

WHAT THIS TERM MEASURES

The difference in means tells you whether the centres of the two distributions have drifted apart.

EXAMPLE

Why the mean alone is not enough

Now suppose both distributions are centred in exactly the same place, but one is far more concentrated than the other.

Real: $\mu = 10, \sigma = 20$

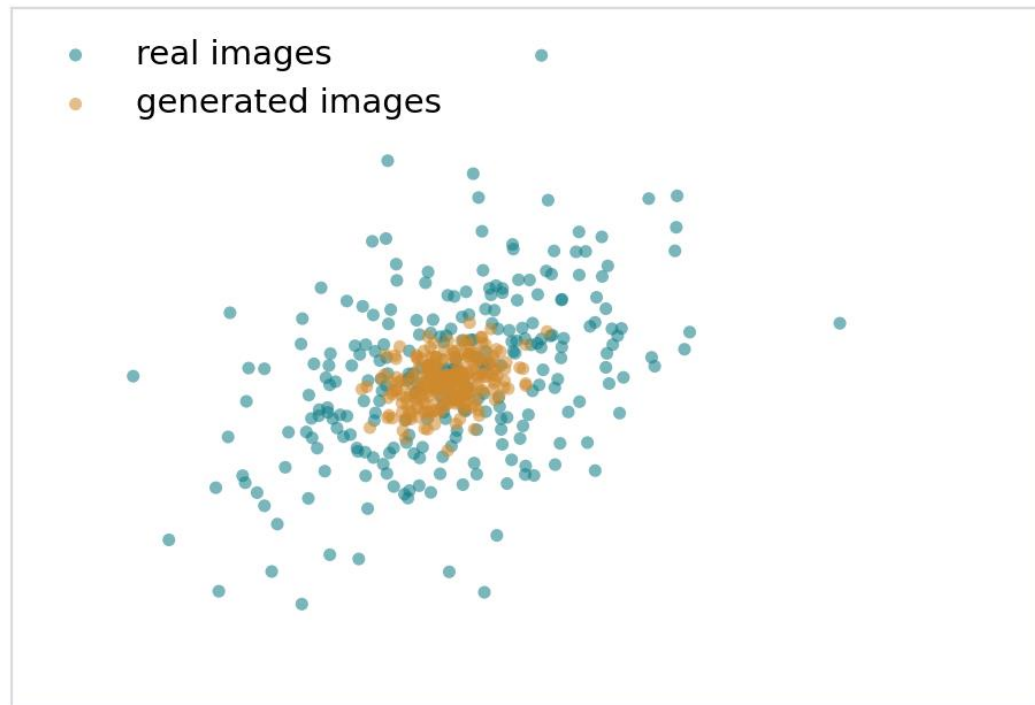
Generated: $\mu = 10, \sigma = 2$

Identical means — yet these are clearly not the same distribution. So FID also needs covariance, which in many dimensions describes both the spread and the relationships between feature dimensions.

MODE COLLAPSE, DRAWN IN FEATURE SPACE

The generated cloud sits on the real one but fills almost none of it.

Bad generator: too little spread



THE FID FORMULA

DIFFERENCE IN CENTRES

$$\|\mu_r - \mu_g\|^2$$

How far apart are the two clouds?

DIFFERENCE IN SPREAD AND SHAPE

$$\text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

Do they occupy the same volume, the same way?

FID $\approx$ difference in centre + difference in spread and shape

ALGORITHM

Computing FID in practice

- 1 Pass all 10,000 real images through the Inception network.
- 2 Extract their feature vectors.
- 3 Compute the real mean and covariance, μ_r and Σ_r .
- 4 Generate 10,000 images and pass them through the same network.
- 5 Extract their feature vectors.
- 6 Compute the generated mean and covariance, μ_g and Σ_g .
- 7 Substitute all four into the FID formula.

THE RESULT

A single number: the FID score for that model, on that dataset, under that protocol.

INTERPRETATION

Reading FID values

Suppose three models are evaluated under an identical protocol.

MODEL A

FID = 8

MODEL B

FID = 25

MODEL C

FID = 75

8 < 25 < 75 → Model A is the best of the three. Lower is better.

WHAT FID = 0 WOULD MEAN

The real and generated feature distributions are identical — a perfect match under this feature representation and Gaussian approximation. In real experiments an exact zero is not expected.

WHY FID IS MORE INFORMATIVE

FID and mode collapse

Return to the failure that defeated the Inception Score.

THE REAL DATASET

cats · dogs · cars · horses · birds

WHAT THE GENERATOR PRODUCES

dogs · dogs · dogs · dogs · dogs

INCEPTION SCORE SAYS

Fine — the dogs are realistic and the classifier is confident.

FID SAYS

Poor — the generated feature distribution is missing most of the real one. FID rises.

Calculating FID by hand

A two-dimensional worked example

The question

The feature distributions of the real and generated images are given below. Calculate the FID.

REAL IMAGES

$$\mu_r = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \quad \Sigma_r = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

GENERATED IMAGES

$$\mu_g = \begin{bmatrix} 2 \\ 3 \end{bmatrix} \quad \Sigma_g = \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix}$$

USING THE FID FORMULA

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}\right)$$

STEP 1

Squared distance between the means

Subtract the two mean vectors, then take the squared length of the difference.

1

$$\mu_r - \mu_g = \begin{bmatrix} 1 \\ 2 \end{bmatrix} - \begin{bmatrix} 2 \\ 3 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$$

$$\|\mu_r - \mu_g\|^2 = (-1)^2 + (-1)^2 = 2$$

This is the first of the two terms in the formula — how far apart the two clouds sit.

STEP 2

Multiply the covariance matrices

The formula needs the square root of the product of the two covariance matrices, so form that product first.

$$2 \quad \Sigma_r \Sigma_g = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix} = \begin{bmatrix} 6 & 0 \\ 0 & 6 \end{bmatrix}$$

Both matrices are diagonal, so the product is diagonal too: multiply the corresponding diagonal entries and leave the zeros alone.

STEP 3

Take the matrix square root

For a diagonal matrix, the square root is the square root of each diagonal entry.

3

$$(\Sigma_r \Sigma_g)^{1/2} = \begin{bmatrix} \sqrt{6} & 0 \\ 0 & \sqrt{6} \end{bmatrix}$$

CHECK IT

$$\begin{bmatrix} 6 & 0 \\ 0 & 6 \end{bmatrix} = \begin{bmatrix} \sqrt{6} & 0 \\ 0 & \sqrt{6} \end{bmatrix} \begin{bmatrix} \sqrt{6} & 0 \\ 0 & \sqrt{6} \end{bmatrix}$$

STEP 4

Assemble the covariance term

Add the two covariance matrices, then subtract twice the square root just computed.

4

$$\Sigma_r + \Sigma_g = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} + \begin{bmatrix} 3 & 0 \\ 0 & 3 \end{bmatrix} = \begin{bmatrix} 5 & 0 \\ 0 & 5 \end{bmatrix}$$

$$\Sigma_r + \Sigma_g - 2 (\Sigma_r \Sigma_g)^{1/2} = \begin{bmatrix} 5 - 2\sqrt{6} & 0 \\ 0 & 5 - 2\sqrt{6} \end{bmatrix}$$

STEP 5

Take the trace

The trace is the sum of the diagonal entries.

$$5 \quad \text{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}\right) = (5 - 2\sqrt{6}) + (5 - 2\sqrt{6}) = 10 - 4\sqrt{6} \approx 0.2020$$

THE COVARIANCE TERM

$$10 - 4\sqrt{6} \approx 0.2020$$

Small — because the two spreads were already quite similar.

STEP 6

Add the two terms

The FID is the mean term plus the covariance term.

MEAN TERM

$$\|\mu_r - \mu_g\|^2 = 2$$

COVARIANCE TERM

$$\text{Tr}(\dots) = 10 - 4\sqrt{6}$$

6

$$\text{FID} = 2 + (10 - 4\sqrt{6}) = 12 - 4\sqrt{6} \approx 2.2020$$

FINAL ANSWER

$$\text{FID} = 12 - 4\sqrt{6} \approx 2.20$$

MEAN TERM

$$\|\mu_r - \mu_g\|^2 = 2$$

COVARIANCE TERM

$$\text{Tr}(\dots) \approx 0.2020$$

Almost all of this FID comes from the gap between the centres, not from the difference in spread.

DISCUSSION

What to notice about this example

WHY DIAGONAL MATRICES

The covariances here are diagonal — so the matrix square root is just the square root of each diagonal entry.

THE GENERAL CASE IS HARDER

With real Inception features the covariances are dense 2048×2048 matrices. The square root of their product needs a numerical routine, and that step dominates the cost of computing FID.

READING THE RESULT

Both terms are non-negative. FID is zero only when the means and the covariances match exactly.

Two caveats to state with the number

The Gaussian assumption is a simplification

FID summarises each feature cloud by a mean and a covariance. Real Inception features are not Gaussian, so the number is a useful comparison — not a distance between the true distributions.

FID is biased when you have few samples

Fewer samples inflate FID. The convention is 50 000 generated images against the full real set. With a few hundred images, report KID instead.

Inception Score vs FID

	Inception Score	FID
What it reads	the 1000-way softmax	the 2048-d pooling features
Uses your real data?	no	yes
Direction	higher is better	lower is better
Mode collapse	largely missed	detected

One row explains all the others: FID compares against your data, IS never sees it.

What to report in practice

- Report FID as the headline number. Keep IS as a secondary one, if at all.
- Hold the protocol fixed: same sample count, same Inception weights, same preprocessing for every model you compare.
- Never compare your FID with a number from a paper unless the protocol matches.

And keep looking at the samples. No single number replaces a contact sheet of generated images.

BEYOND A SINGLE NUMBER

Improved Precision & Recall for GANs

FID collapses quality and coverage into one scalar — it tells you *that* a generator is worse, never *how*.

PRECISION — QUALITY

What fraction of the generated images look real?

What fraction of generated samples fall inside the manifold of the real data.

RECALL — DIVERSITY

What fraction of the real data can the generator produce?

What fraction of real samples fall inside the manifold of the generated data.

WHY TWO NUMBERS

The two failures need opposite fixes.

A truncated generator needs more diversity; an under-trained one needs more fidelity.

Kynkäänniemi, Karras, Laine, Lehtinen & Aila (2019). “Improved Precision and Recall Metric for Assessing Generative Models.” NeurIPS 32.

MOTIVATION

Two generators, one FID

A single distance can be blind to the difference between these two models.

GENERATOR A

Truncated / mode-collapsed

- Every sample is sharp and convincing.
- It only ever draws one kind of image — half the dataset is never produced.
- Fix: more diversity — weaken truncation, add minibatch std-dev.

high precision · low recall

GENERATOR B

Under-trained / blurry

- Every mode of the data is represented.
- But every sample is soft and smeared — none would pass as real.
- Fix: more fidelity — train longer, sharpen the generator.

low precision · high recall

THE DEFINITION, IN PICTURES

Precision and recall for distributions

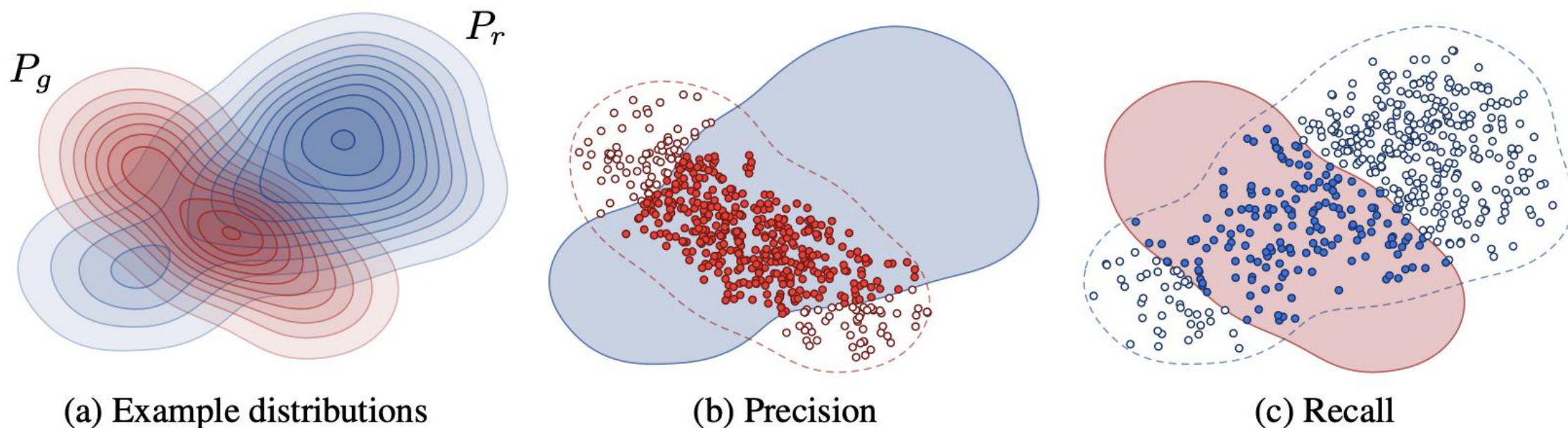


Figure 1: Definition of precision and recall for distributions [24]. (a) Denote the distribution of real images with P_r (blue) and the distribution of generated images with P_g (red). (b) Precision is the probability that a random image from P_g falls within the support of P_r . (c) Recall is the probability that a random image from P_r falls within the support of P_g .

An Example

REAL SAMPLES Φ_r

SAMPLE	$\phi = (f_1 , f_2)$
r1	(1, 1)
r2	(2, 1)
r3	(1.5, 2)
r4	(5, 5)
r5	(6.5, 5)

GENERATED SAMPLES Φ_g

SAMPLE	$\phi = (f_1 , f_2)$
g1	(1.2, 1.1)
g2	(1.8, 1.2)
g3	(1.4, 1.6)
g4	(2.2, 1.5)
g5	(3.5, 3)

Each image has 2 features

STEP 1 — THE BUILDING BLOCK

Euclidean distance in feature space

Every quantity in this metric is built from one operation: the straight-line distance between two feature vectors.

THE FORMULA

$$\|\varphi_a - \varphi_b\|_2 = \sqrt{\sum (\varphi_a[i] - \varphi_b[i])^2}$$

Summed over every dimension i . With VGG-16 features that is $i = 1 \dots 4096$; in our toy example it is just two.

A CLOSE PAIR

$$d(r1, g1) \quad r1 = (1.0, 1.0), \quad g1 = (1.2, 1.1)$$

$$\begin{aligned} \Delta &= (1.2 - 1.0, \quad 1.1 - 1.0) \\ &= (0.2, \quad 0.1) \end{aligned}$$

$$d = \sqrt{(0.2^2 + 0.1^2)} = \sqrt{0.05}$$

$$= \mathbf{0.224} \quad \text{nearly identical}$$

A DISTANT PAIR

$$d(r4, g5) \quad r4 = (5.0, 5.0), \quad g5 = (3.5, 3.0)$$

$$\begin{aligned} \Delta &= (3.5 - 5.0, \quad 3.0 - 5.0) \\ &= (-1.5, \quad -2.0) \end{aligned}$$

$$d = \sqrt{(1.5^2 + 2.0^2)} = \sqrt{6.25}$$

$$= \mathbf{2.500} \quad \text{far apart — remember this}$$

STEP 2 — THE RADIUS

How big is each sample's hypersphere?

Each sample gets a sphere reaching exactly to its k -th nearest neighbour **inside its own set**.

THE RADIUS FORMULA

$$r_k(\varphi, \Phi) = \|\varphi - \text{NN}_k(\varphi, \Phi)\|_2$$

NN_k = the k -th nearest neighbour of φ within the set Φ .

WORKED: RADIUS OF $r_4 = (5.0, 5.0)$

distances to the other real samples:

$r_5 \rightarrow 1.500$ $r_2 \rightarrow 5.000$ $r_3 \rightarrow 4.950$ $r_1 \rightarrow 5.657$

Nearest ($k = 1$) is r_5 , so $r_1(r_4) = d(r_4, r_5) = \sqrt{(1.5^2 + 0^2)} = \mathbf{1.500}$

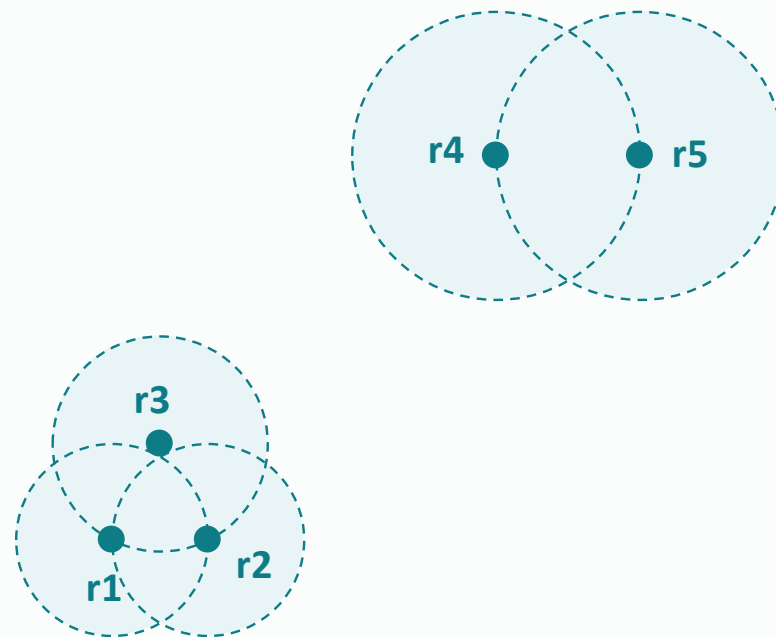
WORKED: RADIUS OF $r_3 = (1.5, 2.0)$

distances to the other real samples:

$r_1 \rightarrow 1.118$ $r_2 \rightarrow 1.118$ $r_4 \rightarrow 4.950$ $r_5 \rightarrow 5.831$

Nearest ($k = 1$) is r_1 , so $r_1(r_3) = d(r_3, r_1) = \sqrt{(0.5^2 + 1.0^2)} = \mathbf{1.118}$

THE REAL MANIFOLD



Union of the dashed circles = the manifold.

STEP 3 — THE MAIN FORMULA

The binary membership test

$$f(\varphi, \Phi) = 1 \text{ if } \|\varphi - \varphi'\|_2 \leq \|\varphi' - \text{NN}_k(\varphi', \Phi)\|_2 \text{ for at least one } \varphi' \in \Phi$$
$$f(\varphi, \Phi) = 0 \text{ otherwise}$$

φ	the point being tested	One image's feature vector. For precision this is a generated sample; for recall it is a real one.
Φ	the reference set	The set whose manifold we are testing against — the opposite set from ϕ .
$\ \varphi - \varphi'\ _2$	left-hand side	How far our point is from some member ϕ' of the reference set. Euclidean distance, from Step 1.
$\ \varphi' - \text{NN}_k(\varphi', \Phi)\ _2$	right-hand side	That member's own radius, from Step 2. Its k-th neighbour distance.
at least one	the quantifier	One sphere is enough. We scan the whole reference set and stop at the first hit.

STEP 4 — AVERAGE THE TEST

Precision and recall

Run the membership test over every sample in one set and take the mean. That is all that is left.

PRECISION

$$\text{precision} = 1 / |\Phi_g| \cdot \sum f(\varphi_g, \Phi_r)$$

Loop over **generated** samples; the spheres belong to the **real** set.

RECALL

$$\text{recall} = 1 / |\Phi_r| \cdot \sum f(\varphi_r, \Phi_g)$$

Loop over **real** samples; the spheres belong to the **generated** set.

HOW TO TELL THEM APART

Do not memorise the shape — **read the arguments of f**. Sum over $\varphi_g \rightarrow$ precision. Sum over $\varphi_r \rightarrow$ recall.

Because f is binary, both numbers are automatically between 0 and 1 — **a plain percentage of samples**, unlike FID's uninterpretable scale.

WORKED EXAMPLE

Five real images, five generated

Pretend VGG-16 gives only two features: f_1 = stroke thickness, f_2 = curvature. Here $k = 1$.

REAL Φ_r

mode A — thin & round

r1(1.0, 1.0) r2(2.0, 1.0)

r3(1.5, 2.0)

mode B — thick & angular

r4(5.0, 5.0) r5(6.5, 5.0)

GENERATED Φ_g

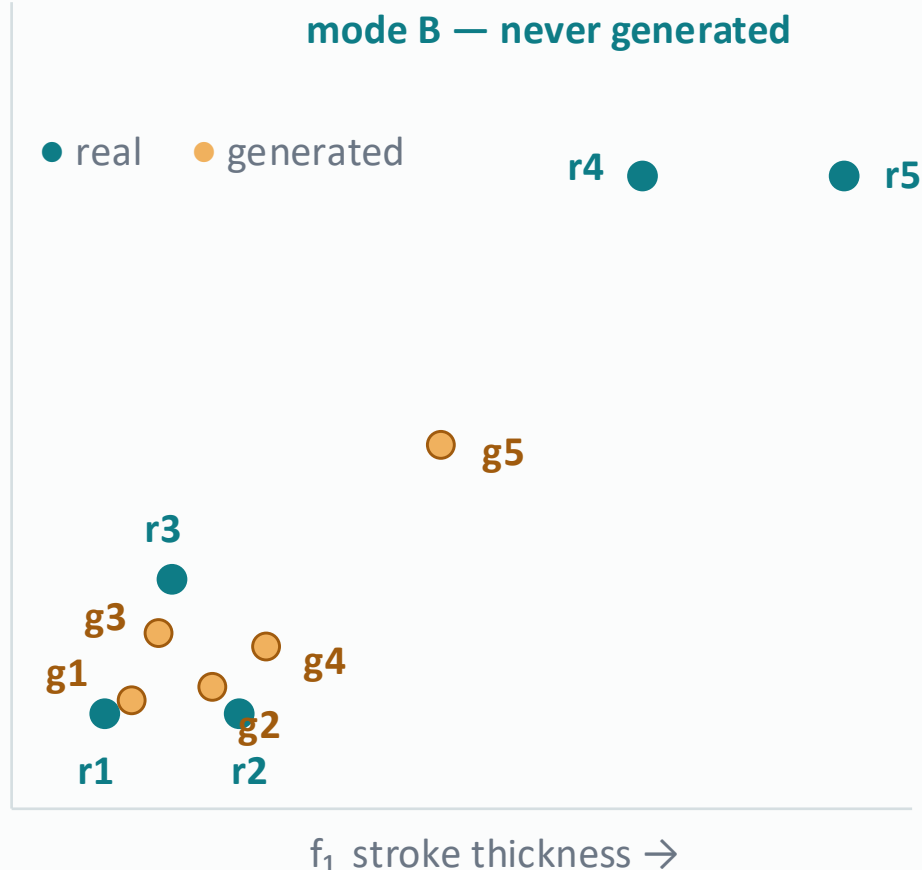
g1(1.2, 1.1) g2(1.8, 1.2)

g3(1.4, 1.6) g4(2.2, 1.5)

g5(3.5, 3.0)

Predict both numbers first: four of five generated samples sit on mode A.

FEATURE SPACE



STEP 1 · BUILD THE TWO MANIFOLDS

Every radius, computed

$r_1(\varphi)$ = distance to the nearest other sample **in the same set**.

REAL SAMPLES Φ_r

SAMPLE	$\phi = (f_1 , f_2)$	NEAREST	RADIUS
r1	(1, 1)	r2	1.000
r2	(2, 1)	r1	1.000
r3	(1.5, 2)	r1	1.118
r4	(5, 5)	r5	1.500
r5	(6.5, 5)	r4	1.500

GENERATED SAMPLES Φ_g

SAMPLE	$\phi = (f_1 , f_2)$	NEAREST	RADIUS
g1	(1.2, 1.1)	g3	0.539
g2	(1.8, 1.2)	g4	0.500
g3	(1.4, 1.6)	g1	0.539
g4	(2.2, 1.5)	g2	0.500
g5	(3.5, 3)	g4	1.985

WHAT THE TWO COLUMNS ALREADY TELL YOU

Generated radii are **roughly half** the real ones — the samples are packed together. The exception is **g5**, stranded on its own, which gets a radius of 1.985. Remember that one.

STEP 2 · PRECISION

Test each generated sample against the real manifold

For each g : is there any real sample r with $d(g, r) \leq \text{radius of } r$?

GENERATED SAMPLE	CLOSEST COVERING REAL	DISTANCE d	ITS RADIUS r_1	$d \leq r_1$?
g1 (1.2, 1.1)	r1	0.224	1.000	✓ inside
g2 (1.8, 1.2)	r2	0.283	1.000	✓ inside
g3 (1.4, 1.6)	r3	0.412	1.118	✓ inside
g4 (2.2, 1.5)	r2	0.539	1.000	✓ inside
g5 (3.5, 3)	r4	2.500	1.500	✗ outside

PRECISION

4 / 5 = 0.800

of generated samples lie on the real manifold

Only **g5** fails — 2.500 from its closest real neighbour r4, whose radius is 1.500. **The images this GAN makes are, on the whole, convincing.**

k = 1 in this toy example; the paper uses k = 3 on 50 000 samples.

STEP 3 · RECALL

Now swap the roles: test each real sample

For each r : is there any generated sample g with $d(r, g) \leq \text{radius of } g$?

REAL SAMPLE	CLOSEST GENERATED	DISTANCE d	ITS RADIUS r_1	$d \leq r_1$?
r1 (1, 1)	g1	0.224	0.539	✓ inside
r2 (2, 1)	g2	0.283	0.500	✓ inside
r3 (1.5, 2)	g3	0.412	0.539	✓ inside
r4 (5, 5)	g5	2.500	1.985	✗ outside
r5 (6.5, 5)	g5	3.606	1.985	✗ outside

RECALL

3 / 5 = 0.600

of real samples are reachable by the generator

Both of **mode B** fail — 2.500 and 3.606 from the nearest generated sample, whose radius is 1.985. **A whole mode of the dataset does not exist for this GAN.**

READING THE RESULT

What the two numbers told us that FID could not

PRECISION

0.800

Quality is good — most samples look real.

RECALL

0.600

Coverage is poor — a whole mode is missing.

DIAGNOSIS

Mode collapse

Not an image-quality problem at all.

WHAT A SINGLE FID WOULD HAVE SAID

“This model is mediocre.” It would not have said **which** of the two failure modes you are looking at — and they need opposite fixes.

SO WHAT DO YOU ACTUALLY DO NEXT?

Do not train longer for sharper images — precision is already fine. **Attack the coverage:** weaken truncation, add minibatch standard deviation, check for a discriminator that has overpowered the generator, or rebalance an unbalanced dataset.

IN PRACTICE

How the paper actually computes it

SETTINGS THAT MATTER

Feature space

VGG-16, second fully connected layer — 4096-d.

Neighbours

$k = 3$. Larger k covers more, but inflates the manifold.

Sample count

50 000 per distribution, as with FID.

Distance

Plain Euclidean distance, exactly as in Step 1.

What to report

Both numbers, with your k and sample count stated.

THE EVIDENCE IT WORKS: TRUNCATION

Turn StyleGAN's ψ down and samples get cleaner but less varied. A correct metric must show precision rising and recall falling.

THIS METRIC

Precision $\uparrow$, recall $\downarrow$ as $\psi \rightarrow 0$. Matches what everyone sees.

SAJJADI ET AL. (2018)

Reports both rising together — confused by tightly packed samples.

The authors used the metric to **re-tune StyleGAN itself** — dropping instance normalisation raised recall and reached state-of-the-art FID.

Resources

- **GAN Paper:** Goodfellow, Ian J., et al. "Generative adversarial nets." *Advances in neural information processing systems* 27 (2014).
• https://papers.nips.cc/paper_files/paper/2014/hash/f033ed80deb0234979a61f95710dbe25-Abstract.html
- **Inception Score Paper (Section 4):** Salimans, Tim, et al. "Improved techniques for training gans." *Advances in neural information processing systems* 29 (2016).
• https://proceedings.neurips.cc/paper_files/paper/2016/file/8a3363abe792db2d8761d6403605aeb7-Paper.pdf
- **FID measure Paper:** Heusel, Martin, et al. "Gans trained by a two time-scale update rule converge to a local nash equilibrium." *Advances in neural information processing systems* 30 (2017).
• https://proceedings.neurips.cc/paper_files/paper/2017/file/8a1d694707eb0fefe65871369074926d-Paper.pdf
- **Improved Precision & Recall Paper:** Kynkäänniemi, Tuomas, et al. "Improved precision and recall metric for assessing generative models." *Advances in neural information processing systems* 32 (2019).
• https://proceedings.neurips.cc/paper_files/paper/2019/file/0234c510bc6d908b28c70ff313743079-Paper.pdf